

דוח פרויקט: חיזוי מחלות לב

מגישים: יונתן גבאי 211847215 ונועה הוניגשטיין 329808554

הערה: מודל Random Forest ומודל SVM (מודלים חמישי ושישי) נוספו לאחר הצגת הפרויקט.

1. על מאגר הנתונים

בפרויקט הזה עבדנו עם [Heart Disease Dataset](#) מתוך Kaggle. המאגר כולל נתונים רפואיים של מטופלים, והמטרה היא לחזות אם קיימת מחלת לב: יש מחלה, 0 = אין מחלה = 1.

בנתונים יש 13 מאפיינים רפואיים עיקריים, למשל גיל, כולסטרול, לחץ דם, סוג כאב חזה, קצב לב מקסימלי ועוד.

עמודה	תיאור
age	גיל
sex	מין (זכר, 0=נקבה=1)
cp	סוג כאב חזה (0-3)
trestbps	לחץ דם במנוחה (mm Hg)
chol	כולסטרול בדם (mg/dl)
fbs	סוכר בצום (> 120 mg/dl)
restecg	תוצאת ECG במנוחה
thalach	קצב לב מקסימלי
exang	אנגינה בעקבות מאמץ
oldpeak	ירידת ST במאמץ
slope	שיפוע מקטע ST
ca	מספר כלי דם ראשיים
thal	תלסמיה

2. מה רצינו לבדוק

במהלך העבודה שאלנו שלוש שאלות מרכזיות:

- האם אפשר לחזות מחלת לב על סמך הנתונים הרפואיים?
- איזה מודל עובד הכי טוב על המאגר הזה?
- אילו מאפיינים משפיעים הכי הרבה על התחזית?

3. עיבוד מקדים של הנתונים

3.1 הסרת כפילויות

אחד הדברים הראשונים שראינו הוא שיש במאגר שורות כפולות. זה חשוב, כי אם אותה שורה מופיעה גם באימון וגם בבדיקה, נוצר מצב של Data Leakage – כלומר, המודל בעצם כבר "פגש" את הדוגמה קודם.

כדי לבדוק את ההשפעה של זה, הרצנו את המודלים גם עם כפילויות וגם בלי כפילויות. ראינו שהפער הכי בולט היה ב-1-NN: עם כפילויות הוא הגיע ל-1.000 דיוק בבדיקה, אבל בלי כפילויות ירד ל-0.770. גם ברשת הניורונים ראינו פער גדול, מ-1.000 ל-0.754. זה חיזק את ההבנה שהכפילויות מנפחות את התוצאה, ולכן מהשלב הזה והלאה עבדנו רק עם נתונים נקיים.

3.2 חלוקה לסט אימון וסט בדיקה

אחרי הניקוי חילקנו את הנתונים ל-80% אימון ו-20% בדיקה, בעזרת Stratified Split. בחרנו בחלוקה הזו כדי לשמור על יחס דומה בין המחלקות גם בסט האימון וגם בסט הבדיקה.

בפועל מימשנו את זה בעצמנו ב-preprocessing.py, ובדקנו שהיחסים באמת נשמרו בעזרת `np.bincount(y_train)` ו-`np.bincount(y_test)`. בסוף קיבלנו 241 דוגמאות אימון ו-61 דוגמאות בבדיקה, עם איזון מחלקות של [110, 131] באימון ו-[28, 33] בבדיקה.

3.3 נרמול הנתונים

בחלק מהמודלים, במיוחד כאלה שמבוססים על מרחק, חשוב מאוד שהמאפיינים יהיו באותה סקאלה. לכן מימשנו `StandardScaler` משלנו ב-preprocessing.py.

הנרמול נעשה לפי הנוסחה:

$$\frac{X_{\text{train}} - \mu}{\sigma_{\text{train}}} = X_{\text{scaled}}$$

נקודה חשובה: את ה-Scaler אימנו רק על סט האימון, ואז הפעלנו אותו על סט הבדיקה. כלומר השתמשנו ב-`fit(X_train)`. ורק אחר כך ב-`transform(X_test)`. עשינו את זה כדי לא להכניס מידע מסט הבדיקה לשלב האימון.

הנרמול היה חשוב במיוחד עבור k-NN (k=5) ועבור הרשת הניורונים. בפועל, k-NN (k=5) השתפר מ-0.705 דיוק בבדיקה בלי נרמול ל-0.869 עם נרמול, והרשת הניורונים השתפרה מ-0.541 ל-0.754. לעומת זאת, עץ ההחלטה נשאר על 0.770 גם עם וגם בלי נרמול, ו-AdaBoost נשאר על 0.902 בשתי הגרסאות.

3.4 אימות צולב

כדי לבחור היפרפרמטרים בצורה אמינה, מימשנו Stratified K-Fold CV עם k=5 ב-evaluation.py.

בכל fold אימנו מחדש את ה-Scaler רק על חלק האימון של אותו fold. זה היה חשוב כדי למנוע גם כאן Data Leakage.

4. מודל ראשון: k-NN

4.1 איך המודל עובד

הרעיון ב-k-NN פשוט: עבור כל דוגמה חדשה, המודל מחפש את k השכנים הכי קרובים בנתוני האימון, ומחזיר את מחלקת הרוב שלהם.

את חישוב המרחקים מימשנו בצורה וקטורית ב-NumPy, בעזרת הזהות:

$$\|b - a\|^2 = \|a\|^2 + \|b\|^2 - 2a \cdot b$$

כך אפשר לחשב מרחקים מהר בלי לעבור בלולאות על כל דוגמה.

4.2 בחירת הערך של k

בדקנו כמה ערכים אפשריים של k:

[1, 3, 5, 7, 9, 11, 15, 21, 31, 41, 51, 61, 71, 81, 91, 111, 131, 151, 181, 211]

בחרנו את הערך שנתן את תוצאת ה-CV הטובה ביותר. במחברת התקבל $best\ k = 11$, עם דיוק CV של 0.809.

הרעיון היה לבדוק גם ערכים קטנים מאוד, שעלולים לגרום ל-overfitting, וגם ערכים גדולים מאוד, שעלולים לגרום ל-underfitting.

4.3 מה ראינו

כשהסתכלנו על גרף של Train Error מול Test Error, היה אפשר לראות את התבנית הצפויה:

- כש- $k=1$, שגיאת האימון כמעט אפסית, אבל שגיאת הבדיקה גבוהה יותר.
- כש-k גדול מדי, המודל כבר נהיה פשוט מדי.
- באמצע מתקבל האיזון הטוב ביותר.

עבור המודל הזה השתמשנו בנתונים המנומרים: X_{train_s} ו- X_{test_s} .

5. מודל שני: עץ החלטה

5.1 איך המודל עובד

מימשנו עץ החלטה מאפס בצורה רקורסיבית. בכל צומת העץ בוחר את הפיצול שנותן את השיפור הכי גדול לפי Information Gain:

$$H(S_R) \frac{|RS|}{|S|} - H(S_L) \frac{|LS|}{|S|} - H(S) = \text{Gain}(S, f, t)$$

בדקנו ספים אפשריים בין ערכים ייחודיים סמוכים של כל מאפיין, ובנוסף חישבנו גם `feature_importances_` לפי התרומה של כל פיצול לאורך העץ.

5.2 בחירת היפרפרמטרים

הרצנו 5-fold CV על כל הצירופים של:

- `max_depth`: [3, 4, 5, 6, 7, 8]
- `min_samples_split`: [10, 12, 15, 18, 20, 25, 30]
- `min_samples_leaf`: [3, 4, 5, 6, 7, 8]

בסוף, השילוב הטוב ביותר היה `max_depth=5`, `min_samples_split=15`, `min_samples_leaf=5`, עם דיוק CV של 0.813 ± 0.045 .

המשמעות של הפרמטרים האלה פשוטה:

- `max_depth` מגביל כמה עמוק העץ יכול לגדול.

- `min_samples_split` מונע פיצול של קבוצות קטנות מדי.
- `min_samples_leaf` דואג שלכל עלה תהיה כמות מינימלית של דוגמאות.

5.3 מה ראינו

ככל שהעץ עמוק יותר, הוא מתאים טוב יותר לסט האימון, אבל לא תמיד לסט הבדיקה. לכן היה חשוב למצוא עומק שלא יהיה גדול מדי. עבור המודל הטוב ביותר קיבלנו 0.867 דיוק על סט האימון ו-0.770 על סט הבדיקה.

בדקנו גם בחירת תכונות לפי `feature_importances_`, כלומר אימנו עץ על כל התכונות, דירגנו אותן, ואז ניסינו להשתמש רק ב-top-1, אחר כך top-2, וכן הלאה. המטרה הייתה לבדוק אם אפשר לשפר ביצועים על ידי הסרת מאפיינים פחות חשובים. במקרה הזה, שלוש התכונות המובילות היו `ca`, `cp` ו-`slope`. עם שלושתן קיבלנו CV accuracy = 0.826 במקום 0.813 עם כל 13 התכונות, ודיוק הבדיקה עלה מ-0.770 ל-0.787.

6. מודל שלישי: AdaBoost

6.1 איך המודל עובד

AdaBoost הוא מודל אנסמבל שבונה הרבה מסווגים פשוטים מאוד, במקרה שלנו Decision Stumps — עצים בעומק 1. בכל סבב המודל:

1. מוצא את ה-stump הטוב ביותר.
2. מחשב את המשקל שלו לפי השגיאה.
3. מעלה את המשקל של דוגמאות שסווגו לא נכון.
4. ממשיך לסבב הבא.

המשקל של כל מסווג חושב לפי:

$$\alpha = 0.5 * \ln((1-\text{error})/\text{error})$$

והניבוי הסופי מתקבל משקלול של כל המסווגים יחד:

$$\text{sign}(\sum \alpha_i * h_i(x))$$

כדי להימנע מבעיות מספריות, השתמשנו גם ב-clipping ל-[1e-10, 1-1e-10].

6.2 בחירת מספר הסבבים

בדקנו את הערכים:

$$[1, 5, 10, 20, 30, 50, 75, 100]$$

גם כאן נעזרנו ב-5-fold CV כדי לבחור את הערך הטוב ביותר. לפי המחברת, הערך שנבחר היה `best n_estimators = 10`, עם דיוק CV של 0.805.

6.3 מה ראינו

ב-AdaBoost לפעמים יותר סבבים באמת משפרים את התוצאה, לפחות עד נקודה מסוימת. לכן הסתכלנו גם על גרף של Train Error מול Test Error, כדי לראות מתי מתחיל overfitting. עבור `n_estimators=10` קיבלנו דיוק בדיקה של 0.787.

בנוסף חישבנו למודל הזה גם `feature_importances_`, לפי סכום התרומות של כל ה-stumps שהשתמשו באותו מאפיין. גם כאן שלוש התכונות המובילות הן `ca`, `cp` ו-`slope`. ב-CV זה שיפר את התוצאה מ-0.805 ל-0.838, אבל על סט הבדיקה לא היה שיפור בפועל: גם עם כל התכונות וגם עם שלוש התכונות הנבחרות התקבל דיוק של 0.787.

7. מודל רביעי: רשת נוירונים

7.1 איך המודל עובד

מימשנו MLP מאפס. הרשת כללה:

- שכבות חבויות עם ReLU או tanh
- שכבת פלט עם Sigmoid
- פונקציית Loss מסוג Binary Cross-Entropy
- L2 regularization
- אופטימיזציה עם Mini-batch Gradient Descent

בנוסף השתמשנו באתחול משקולות מתאים: He initialization עבור ReLU ו-Xavier עבור tanh.

7.2 בדיקת גרדיאנט

לפני האימון, רצינו לוודא שה-Backpropagation שלנו באמת נכון. לכן עשינו Gradient Check והשוונו בין גרדיאנט אנליטי לגרדיאנט מספרי.

הבדיקה נעשתה לפי:

$$\frac{\|g - \hat{g}\|}{\|g\| + \|\hat{g}\|} = \text{relative diff}$$

כאשר $\hat{g}$ הוא הגרדיאנט המספרי ו- g הוא הגרדיאנט האנליטי. מבחינתנו, תוצאה של $1e-5 <$ נחשבה הצלחה. בפועל התקבלה תוצאה מצוינת: $5.02e-10$.

7.3 עצירה מוקדמת

כדי למנוע overfitting, הקצנו 10% מנתוני האימון כ-validation set. אם ה-validation loss לא השתפר במשך 20 epochs רצופים, עצרנו את האימון וחזרנו למשקולות הטובות ביותר.

7.4 בחירת היפרפרמטרים

הרצנו 5-fold CV על כמה ארכיטקטורות וכמה ערכים של קצב למידה ו-L2:

- `hidden_layers`: (8,), (16,), (32,), (64,), (16,8), (32,16)
- `lr`: 0.1, 0.01
- `l2`: 1e-4, 1e-3, 1e-5

אם היו כמה אפשרויות עם תוצאה דומה, העדפנו רשת קטנה ופשוטה יותר. בפועל, השילוב שקיבל את תוצאת ה-CV הטובה ביותר היה `hidden_layers=(64,)`, `l2=1e-4`, `lr=0.1`, עם 0.801 ± 0.051 .

7.5 מה ראינו

בדקנו גם איך מספר ה-epochs משפיע על המודל, בעזרת הערכים:

[10, 25, 50, 100, 200, 300, 400, 500]

כך היה אפשר לראות מתי המודל עדיין לומד, ומתי הוא כבר מתחיל להתאים את עצמו יותר מדי לנתוני האימון. בריצה הסופית עם Early Stopping האימון נעצר אחרי 26 אפוקים, כשה-best epoch היה 5, ודיוק הבדיקה של המודל הסופי היה 0.852.

8. מודל חמישי: Random Forest

נוסף לאחר הצגת הפרויקט.

8.1 איך המודל עובד

Random Forest הוא מודל אנסמבל שבונה הרבה עצי החלטה ומשלב את התחזיות שלהם. שני מקורות של אקראיות גורמים לעצים להיות שונים זה מזה:

1. כל עץ מאומן על מדגם Bootstrap של השורות (דגימה עם החזרה).
2. בכל פיצול, העץ בוחר את המאפיין הטוב ביותר רק מתוך תת-קבוצה אקראית של מאפיינים `(max_features="sqrt")`.

בזכות זה העצים פחות מתואמים זה עם זה, והממוצע של התחזיות שלהם מקטין את השונות (Variance) בהשוואה לעץ בודד. כדי לממש את האקראיות הזו הוספנו תמיכה בפרמטר `max_features` למחלקת עץ ההחלטה הקיימת. המודל מבוסס-עצים, ולכן אינו רגיש לסקאלה של המאפיינים, וניתן לחשב עבורו `feature_importances_` כממוצע החשיבויות של כל העצים.

8.2 בחירת היפרפרמטרים

הרצנו 5-fold CV על צירופים של מספר העצים והעומק המרבי:

- `n_estimators`: [50, 100, 200]
- `max_depth`: [4, 6, None]

השילוב הטוב ביותר היה `n_estimators=50` ו-`max_depth=6`, עם דיוק CV של 0.809 ± 0.021 . עבור המודל הסופי קיבלנו דיוק אימון של 0.975 ודיוק בדיקה של 0.869.

8.3 מה ראינו

בדקנו את גרף ה-Train Error מול ה-Test Error כתלות במספר העצים. בניגוד לעץ בודד, הוספת עצים ליער כמעט אף פעם לא גורמת ל-`overfitting`: שגיאת הבדיקה ירדה מ-0.213 עם עץ אחד ל-0.115 עם 200 עצים ואז התייצבה, בעוד ששגיאת האימון התאפסה במהירות.

בדקנו גם בחירת תכונות לפי `feature_importances_`. מעניין לראות שביער החשיבות מתפזרת בין יותר מאפיינים (כי כל פיצול רואה רק תת-קבוצה אקראית שלהם), ולכן דירוג המאפיינים שטוח יותר מאשר בעץ בודד. בבחירה הטובה ביותר נבחרו 11 המאפיינים המובילים, ודיוק הבדיקה עם תת-הקבוצה עלה מ-0.869 ל-0.902.

9. מודל שישי: SVM

נוסף לאחר הצגת הפרויקט.

9.1 איך המודל עובד

מימשנו מאפס Support Vector Machine עם שוליים רכים (Soft-Margin) וגרעין RBF. ה-SVM מחפש את המפריד שממקסם את השוליים בין המחלקות, והגרעין מאפשר הפרדה לא-ליניארית על ידי מיפוי מרומז למרחב ממימד גבוה:

$$\exp(-\gamma \|x - z\|^2) = K(x, z)$$

את בעיית האופטימיזציה פתרנו עם גרסה מפושטת של אלגוריתם SMO, שמעדכן בכל צעד זוג מכפילי Lagrange (α) עד להתכנסות. כמו k-NN והרשת הניורונים, זהו מודל מבוסס-מרחק, ולכן חשוב מאוד להזין לו נתונים מנורמלים (z-score).

9.2 בחירת היפרפרמטרים

הרצנו 5-fold CV על צירופים של פרמטר הרגולריזציה C ופרמטר הגרעין γ :

- C: [0.01, 0.1, 1, 10, 100]
- γ : ["scale", 0.01, 0.1, 1]

השילוב הטוב ביותר היה C=1 ו- $\gamma=0.1$, עם דיוק CV של 0.813 ± 0.044 . עבור המודל הסופי קיבלנו דיוק אימון של 0.934 ודיוק בדיקה של 0.869.

9.3 מה ראינו

ההשפעה של הנרמול הייתה בולטת: בלי נרמול ה-SVM הגיע לדיוק בדיקה של 0.639 בלבד, ועם נרמול הוא קפץ ל-0.885.

בגרף ה-Train Error מול ה-Test Error כתלות ב-C ראינו את התבנית הקלאסית של overfitting: כאשר C קטן מאוד (0.01) המודל פשוט מדי ונמצא ב-underfitting (שגיאה 0.459); סביב C=1 מתקבל האיזון הטוב ביותר (שגיאת בדיקה 0.131); וכאשר C גדול מדי (100) המודל מתאים את עצמו יתר על המידה לאימון (שגיאת האימון מתאפסת אך שגיאת הבדיקה עולה ל-0.164).

10. השוואה בין המודלים

אחרי שסיימנו לכוון את כל המודלים, אימנו כל אחד מהם עם ההגדרות הטובות ביותר שמצאנו.

מודל	נרמול	מאפיינים
k-NN (k=11)	כן	כל המאפיינים
Decision Tree (depth=5, split=15, leaf=5)	לא	cp, ca, slope
AdaBoost (n_estimators=10)	לא	cp, ca, slope
Random Forest (n_estimators=50, max_depth=6)	לא	11 תכונות מובילות
SVM (RBF, C=1, gamma=0.1)	כן	כל המאפיינים
Neural Network ((64,), lr=0.1, l2=1e-4)	כן	כל המאפיינים

את התוצאות סיכמנו בצורה הבאה:

```
results_df:
  train acc  test acc  overfit gap
Random Forest    0.975    0.902    0.073
```

k-NN	0.855	0.885	-0.030
SVM	0.934	0.869	0.065
Neural Net	0.842	0.852	-0.010
Decision Tree	0.838	0.787	0.051
AdaBoost	0.838	0.787	0.051

מהטבלה רואים שלאחר הוספת שני המודלים החדשים, המודל הטוב ביותר היה Random Forest עם דיוק בדיקה של 0.902. אחריו הגיע k-NN עם 0.885, אחריו SVM עם 0.869, ואז הרשת הניורונים עם 0.852. עץ ההחלטה ו-AdaBoost נשארו מאחור עם 0.787 כל אחד. הפער בין train accuracy ל-test accuracy עזר לנו להבין עד כמה כל מודל עושה overfit.

11. חשיבות המאפיינים

את ניתוח החשיבות עשינו בעזרת עץ ההחלטה, AdaBoost ו-Random Forest, כי בשלושת המודלים האלה אפשר לחשב `feature_importances_` בצורה ישירה יחסית.

נרמלנו את וקטורי החשיבות כך שהסכום שלהם יהיה 1, ואז הסתכלנו גם על הממוצע בין שלושת המודלים.

ראינו שיש כמה מאפיינים שחוזרים שוב ושוב כחשובים. לפי ממוצע החשיבות של שלושת המודלים, שלושת המאפיינים הבולטים ביותר היו `cp` עם 0.221, אחריו `ca` עם 0.144, ואז `slope` עם 0.106. אחריהם הגיעו `thalach` עם 0.084 ו-`thal` עם 0.074.

12. קשיים עיקריים בדרך

כפילויות במאגר

הכפילויות יצרו תמונה מטעה של ביצועים טובים יותר ממה שבאמת קיבלנו. הפתרון היה להסיר אותן ולעבוד רק עם נתונים נקיים.

דליפת מידע בנרמול

היה חשוב לא לאמן את ה-Scaler על כל הנתונים יחד. לכן הקפדנו שהוא ילמד רק מ-`X_train`.

דליפת מידע ב-Cross Validation

גם בתוך ה-CV, בכל fold אימנו מחדש את ה-Scaler רק על נתוני האימון של אותו סבב.

מימוש נכון של Backpropagation

כדי להיות בטוחים שהרשת הניורונים באמת מומשת נכון, השתמשנו ב-Gradient Check.

זמן ריצה

במיוחד בעץ ההחלטה, מספר הצירופים שבדקנו היה גדול. למרות זאת, זמן הריצה עדיין היה סביר ואפשר היה לעקוב אחרי ההתקדמות.

13. סיכום

האם אפשר לחזות מחלת לב?

כן. כל ששת המודלים הצליחו להגיע לתוצאות טובות יותר מניחוש אקראי, ולכן אפשר לומר שיש במאגר מספיק מידע כדי לבנות מודל חיזוי שימושי. דיוקי הבדיקה במחברת נעו בין 0.787 ל-0.902.

איזה מודל היה הכי טוב?

לפי התוצאות הסופיות במחברת, ולאחר הוספת שני המודלים החדשים, המודל הטוב ביותר היה Random Forest, עם דיוק בדיקה של 0.902. אחריו הגיע k-NN עם 0.885, אחריו SVM עם 0.869, ואז הרשת הנירונים עם 0.852, ולבסוף עץ ההחלטה ו-AdaBoost עם 0.787 כל אחד. מצד שני, עץ החלטה הוא עדיין המודל שהכי קל להסביר ולהבין.

אילו מאפיינים היו הכי חשובים?

מהניתוח שלנו עלה שהמאפיינים המרכזיים ביותר היו **cp**, **ca** ו-**slope**. אלה גם בדיוק שלוש התכונות שנבחרו גם בעץ ההחלטה וגם ב-AdaBoost בשלב בחירת המאפיינים.

בסך הכול, הפרויקט הראה שלא רק שאפשר לחזות מחלת לב בצורה סבירה, אלא גם שאפשר להבין אילו גורמים משפיעים יותר על התחזית.

14. קישורים

- קוד: heart_disease.ipynb ו-src
- מאגר הקוד ב-GitHub: github.com/yonatang8675/ML_Project
- מאגר הנתונים: [Kaggle - Heart Disease Dataset](https://www.kaggle.com/heart)